

MINDSCAN

AI-Powered Mental Health Sentiment Analyzer

Project Report

Introduction to Artificial Intelligence
4th Semester

Instructor

Ma'am Asia

Team Members

Syeda Fatima Zahra	240201009
Fatima Anjum	240201033
Anosha Shams	240201041

Computer Science Department
Institute of Space Technology (IST)

1. Project Overview

MindScan is an NLP-based mental health sentiment classifier designed to analyze free-form text input and predict which of seven mental health categories the text most closely reflects. The user types how they are feeling, and the system returns a predicted category, a confidence score, a tailored empathy message, and relevant Pakistani mental health helpline resources.

This project was developed as a complete end-to-end machine learning pipeline, covering all major concepts from the Introduction to Artificial Intelligence course: dataset acquisition, exploratory data analysis (EDA), text preprocessing, model training with multiple classifiers, evaluation, explainability, and deployment via a web application.

1.1 Objectives

- Build a multi-class text classifier for mental health sentiment detection.
- Compare traditional ML models (Naive Bayes, Logistic Regression, Random Forest) against a fine-tuned transformer model (DistilBERT).
- Deploy the best-performing model in a Streamlit web application with a user-friendly interface.
- Provide actionable mental health resources based on predicted output.

1.2 Prediction Labels

The classifier predicts one of seven categories:

Label	Description
Anxiety	Worry, panic, nervousness
Bipolar	Mood swings, emotional fluctuation
Depression	Sadness, hopelessness, emptiness
Normal	Stable, healthy mental state
Personality Disorder	Identity, relationship, and emotional regulation issues
Stress	Pressure, overwhelm, burnout
Suicidal	Crisis-level ideation

2. Team Structure

The project was completed by a team of three over a four-week timeline, with responsibilities divided as follows:

Team Member	Role	Primary Deliverable
Fatima Anjum	Data Collection, EDA, and Preprocessing	Clean dataset + EDA Jupyter Notebook
Anosha Shams	Model Training and Evaluation	Trained models + evaluation charts
Fatima Zahra	Web Application and Model Integration	GitHub repository + Streamlit web app

3. Dataset

3.1 Source and Description

Source: Kaggle — Reddit Mental Health NLP Corpus

File: Combined Data.csv

Size: Approximately 50,000 labeled posts (~11 MB)

Labels: 7 categories (Anxiety, Bipolar, Depression, Normal, Personality Disorder, Stress, Suicidal)

Origin: Reddit posts scraped from mental health subreddits and labeled accordingly

Distribution: Shared across team via Google Drive (AIProject shared folder)

3.2 Relevance

The dataset provides real-world, unstructured text representing genuine user expression across a spectrum of mental health conditions. This makes it well-suited for NLP-based classification, as the language patterns are organic and reflect how people naturally describe their mental states. The multi-class nature of the problem (7 labels) also introduces appropriate complexity for a course-level project.

4. Project Roadmap

Phase	Week	Activities
Phase 1 — Data & EDA	Week 1	Load dataset using Pandas; perform EDA with word clouds and class distribution charts; apply NLP preprocessing (cleaning, tokenization, stemming)
Phase 2 — Model Building	Week 2	Train baseline Logistic Regression with TF-IDF (76.29%); train Random Forest and SVM; fine-tune DistilBERT transformer (83% accuracy)

Phase 3 — Evaluation	Week 3	Compute accuracy, F1-score, confusion matrix; generate model comparison bar chart; perform SHAP / LIME feature importance analysis
Phase 4 — Web Application	Week 4	Build Streamlit app; integrate DistilBERT model; set up GitHub repository and documentation

5. Technology Stack

Layer	Technology	Purpose
Language	Python 3.13	All development and scripting
ML Framework	PyTorch	Model inference and training
NLP Library	HuggingFace Transformers	DistilBERT model and tokenizer
Web Framework	Streamlit	UI and app serving
Classical ML	scikit-learn	Naive Bayes, Logistic Regression, Random Forest, TF-IDF
Explainability	LIME	Word-level feature importance (evaluated; removed from final app)
Version Control	Git + GitHub	Source code repository
Training Environment	Google Colab	GPU-backed model fine-tuning
File Sharing	Google Drive	Team file exchange (AIProject folder)

6. Project File Structure

The repository is organized as follows. Model files are excluded from version control due to their size and must be downloaded from the shared Google Drive folder.

```

mental-health-nlp/
├── MindScan/                                ← main project root
│   ├── models/
│   │   ├── distilbert_final/                ← final DistilBERT model (255 MB,
│   │   │   └── gitignored)
│   │   ├── config.json
│   │   ├── model.safetensors
│   │   ├── ...
│   │   └── distilbert_final_tokenizer/      ← tokenizer files (gitignored)
│   │       ├── tokenizer_config.json
│   │       ├── vocab.txt
│   │       └── ...
│   ├── notebooks/
│   │   ├── Combined Data.csv                ← dataset (gitignored)
│   │   └── Untitled0.ipynb                 ← training and evaluation notebook
│   ├── app.py                               ← main Streamlit application
│   └── .streamlit/

```

```

├── config.toml          ← disables file watcher
├── .gitattributes       ← Git LFS tracking rules
├── .gitignore          ← excludes models/, .venv/, *.csv
├── Dockerfile          ← container config for HF Spaces
├── README.md           ← HF Space config + project
├── description
├── requirements.txt    ← Python dependencies
└── .venv/              ← virtual environment (gitignored)

```

6.1 Key Configuration Files

requirements.txt

```

streamlit
torch
transformers
lime
sentencepiece
safetensors
pandas
numpy

```

.streamlit/config.toml

```

[server]
fileWatcherType = "none"

```

This configuration silences hundreds of `ModuleNotFoundError` warnings that Streamlit's default file watcher produces when it scans the Transformers library for torchvision modules.

7. Jupyter Notebook — Code Walkthrough

All model training, evaluation, and analysis was performed in Google Colab. The following subsections document each code cell in the notebook, describing its purpose and the logic behind it.

Cell 1 — Environment Setup and Data Extraction

```

!pip install transformers torch lime -q

import zipfile
import os
import pandas as pd

from google.colab import drive
drive.mount('/content/drive')

with zipfile.ZipFile("/content/drive/MyDrive/AIProject/archive(1).zip", 'r') as zip_ref:

```

```
zip_ref.extractall("/content/mental_health_data")

print(os.listdir("/content/mental_health_data"))
```

Purpose

This cell performs the initial environment setup and data extraction. It installs three required libraries — Transformers (for BERT-based models), PyTorch (the deep learning backend), and LIME (for model explainability) — silently with the `-q` flag to suppress verbose output.

Google Drive is mounted to give the Colab runtime access to the team's shared AIProject folder. The dataset was distributed as a ZIP archive; the `zipfile` module is used to extract its contents into a local Colab directory. The final print statement verifies that extraction succeeded by listing the contents of the target directory.

```
0.0/275.7 kB ? eta -:--:--
275.7/275.7 kB 8.1 MB/s eta 0:00:00

Preparing metadata (setup.py) ... done
Building wheel for lime (setup.py) ... done
Mounted at /content/drive
['Combined Data.csv']
```

Cell 2 — Loading and Inspecting the Dataset

```
import pandas as pd

df = pd.read_csv("/content/mental_health_data/Combined Data.csv")
print(df.shape)
print(df.head())
print(df.columns.tolist())
print(df['status'].value_counts())
```

Purpose

This cell loads the dataset into a Pandas DataFrame and performs an initial inspection. The `shape` attribute shows the total number of rows and columns. The `head()` method previews the first five records to confirm that loading was successful and that the structure is as expected. The `columns` list identifies all available feature names, and `value_counts()` on the target column (`status`) provides a class distribution summary — a critical first step to check for class imbalance before training.

```
(53043, 3)
  Unnamed: 0      statement      status
0          0      oh my gosh  Anxiety
1          1  trouble sleeping, confused mind, restless hear...  Anxiety
2          2  All wrong, back off dear, forward doubt. Stay ...  Anxiety
3          3  I've shifted my focus to something else but I'...  Anxiety
4          4  I'm restless and restless, it's been a month n...  Anxiety
['Unnamed: 0', 'statement', 'status']
status
Normal          16351
Depression      15404
Suicidal        10653
Anxiety         3888
Bipolar         2877
Stress          2669
Personality disorder 1201
Name: count, dtype: int64
```

Cell 3 — Data Cleaning

```
df = df.drop(columns=['Unnamed: 0'])

df = df.dropna(subset=['statement'])

print("Shape after cleaning:", df.shape)
print("Any missing values left?", df.isnull().sum().any())
```

Purpose

This cell handles basic data hygiene. The 'Unnamed: 0' column is a numeric row index artifact that CSV exports often generate automatically; it carries no predictive information and is dropped to avoid confusion. Rows where the statement column is null are also removed, since a missing text input cannot be processed by any model. The post-cleaning shape and null check confirm that the dataset is ready for the preprocessing pipeline.

```
Shape after cleaning: (52681, 2)
Any missing values left? False
```

Cell 4 — Text Preprocessing

```
import re
import nltk
nltk.download('stopwords')
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
```

```

stop_words = set(stopwords.words('english'))
stemmer = PorterStemmer()

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'httpS+', '', text)
    text = re.sub(r'^a-zs]', '', text)
    tokens = text.split()
    tokens = [stemmer.stem(w) for w in tokens if w not in stop_words]
    return ' '.join(tokens)

df['clean_text'] = df['statement'].apply(clean_text)

print(df[['statement', 'clean_text']].head(3))

```

Purpose

This cell defines and applies the full NLP preprocessing pipeline for the classical ML models. The `clean_text` function performs the following steps in sequence:

- Lowercasing — normalizes all text so that 'Anxiety' and 'anxiety' are treated as the same token.
- URL removal — strips hyperlinks using a regular expression, as URLs carry no semantic value for sentiment classification.
- Punctuation and number removal — retains only alphabetic characters and spaces, reducing noise.
- Tokenization — splits the string into individual words.
- Stopword removal — filters out high-frequency words (such as 'the', 'is', 'and') that appear across all classes and do not contribute to distinguishing between them.
- Stemming — reduces words to their root form using the Porter Stemmer (e.g., 'feeling' and 'felt' both become 'feel'), which reduces the vocabulary size and improves generalization.

The function is applied to every row using Pandas' `apply` method, and the result is stored in a new column called `clean_text`. A side-by-side comparison of original and cleaned text is printed for manual verification. Note: this preprocessing is only applied for TF-IDF-based models. DistilBERT uses its own internal tokenizer and does not require this cleaning step.

```

[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]  Unzipping corpora/stopwords.zip.

```

	statement \		clean_text
0	oh my gosh		oh gosh
1	trouble sleeping, confused mind, restless hear...		troubl sleep confus mind restless heart tune
2	All wrong, back off dear, forward doubt. Stay ...		wrong back dear forward doubt stay restless re...

Cell 5 — TF-IDF Vectorization and Train/Test Split

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split

X = df['clean_text']
y = df['status']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

tfidf = TfidfVectorizer(max_features=5000)
X_train_tf = tfidf.fit_transform(X_train)
X_test_tf = tfidf.transform(X_test)

print("Training size:", X_train_tf.shape)
print("Testing size:", X_test_tf.shape)
```

Purpose

Classical machine learning models cannot work directly with raw text; they require numerical input. This cell converts the cleaned text into a numeric matrix using TF-IDF (Term Frequency-Inverse Document Frequency) vectorization.

TF-IDF assigns each word a weight reflecting how frequently it appears in a given document relative to how commonly it appears across all documents. Words that are specific to certain classes receive higher weights, making TF-IDF a strong feature extractor for text classification.

- The dataset is split 80/20 into training and testing sets. `random_state=42` ensures reproducibility across runs.
- `max_features=5000` limits the vocabulary to the 5,000 most informative terms, controlling memory usage and reducing noise from rare words.
- `fit_transform` is called on the training set only, and `transform` (without fitting) is called on the test set. This prevents data leakage — the model must not learn any statistics from the test set during training.

```
Training size: (42144, 5000)
Testing size: (10537, 5000)
```

Cell 6 — Naive Bayes Classifier

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report

nb_model = MultinomialNB()
nb_model.fit(X_train_tf, y_train)
nb_preds = nb_model.predict(X_test_tf)
nb_acc = accuracy_score(y_test, nb_preds)
print(f"Naive Bayes Accuracy: {nb_acc:.2%}")
```

Purpose

This cell trains the first baseline classifier: Multinomial Naive Bayes. Naive Bayes is a probabilistic model based on Bayes' Theorem. It assumes that all features (word counts or TF-IDF scores) are conditionally independent given the class label. While this assumption is rarely true in practice, it tends to work well for text classification because the model captures the overall word-frequency distribution per class effectively.

MultinomialNB is specifically designed for discrete count-like features, making it appropriate for TF-IDF vectors. The model is fast to train, interpretable, and provides a strong performance baseline relative to its simplicity. The accuracy on the test set was 65.95%.

Naive Bayes Accuracy: 65.92%

Cell 7 — Logistic Regression Classifier

```
from sklearn.linear_model import LogisticRegression

lr_model = LogisticRegression(max_iter=1000)
lr_model.fit(X_train_tf, y_train)
lr_preds = lr_model.predict(X_test_tf)
lr_acc = accuracy_score(y_test, lr_preds)
print(f"Logistic Regression Accuracy: {lr_acc:.2%}")
```

Purpose

This cell trains the second baseline classifier: Logistic Regression. Despite its name, Logistic Regression is a classification algorithm. It learns a linear decision boundary in the feature space by fitting a sigmoid (or softmax, for multi-class) function to the training data. Each TF-IDF feature is assigned a learned coefficient that indicates how strongly that word's presence pushes a prediction toward a particular class.

`max_iter=1000` is set to allow the optimization algorithm sufficient iterations to converge on a high-dimensional TF-IDF matrix. Logistic Regression outperformed Naive Bayes in this experiment, achieving 76.29% accuracy, and it was selected as the model saved for early deployment experiments. It was ultimately replaced by DistilBERT in the final application.

Logistic Regression Accuracy: 76.21%

Cell 8 — Random Forest Classifier

```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(n_estimators=100)
rf_model.fit(X_train_tf, y_train)
rf_preds = rf_model.predict(X_test_tf)
rf_acc = accuracy_score(y_test, rf_preds)
print(f"Random Forest Accuracy: {rf_acc:.2%}")
```

Purpose

This cell trains the third classical model: Random Forest. A Random Forest is an ensemble of decision trees, where each tree is trained on a random subset of the training data and a random subset of the features. The final prediction is determined by majority vote across all trees. This ensemble approach reduces variance and overfitting compared to a single decision tree.

`n_estimators=100` builds 100 trees. Random Forest achieved 72.79% accuracy on this dataset — higher than Naive Bayes but lower than Logistic Regression. This is somewhat atypical; it may be attributed to the high-dimensional sparse TF-IDF feature space, which tends to favor linear models like Logistic Regression. Random Forest is considerably slower to train than the other two classical models.

Random Forest Accuracy: 72.80%

Cell 9 — Confusion Matrix (Logistic Regression)

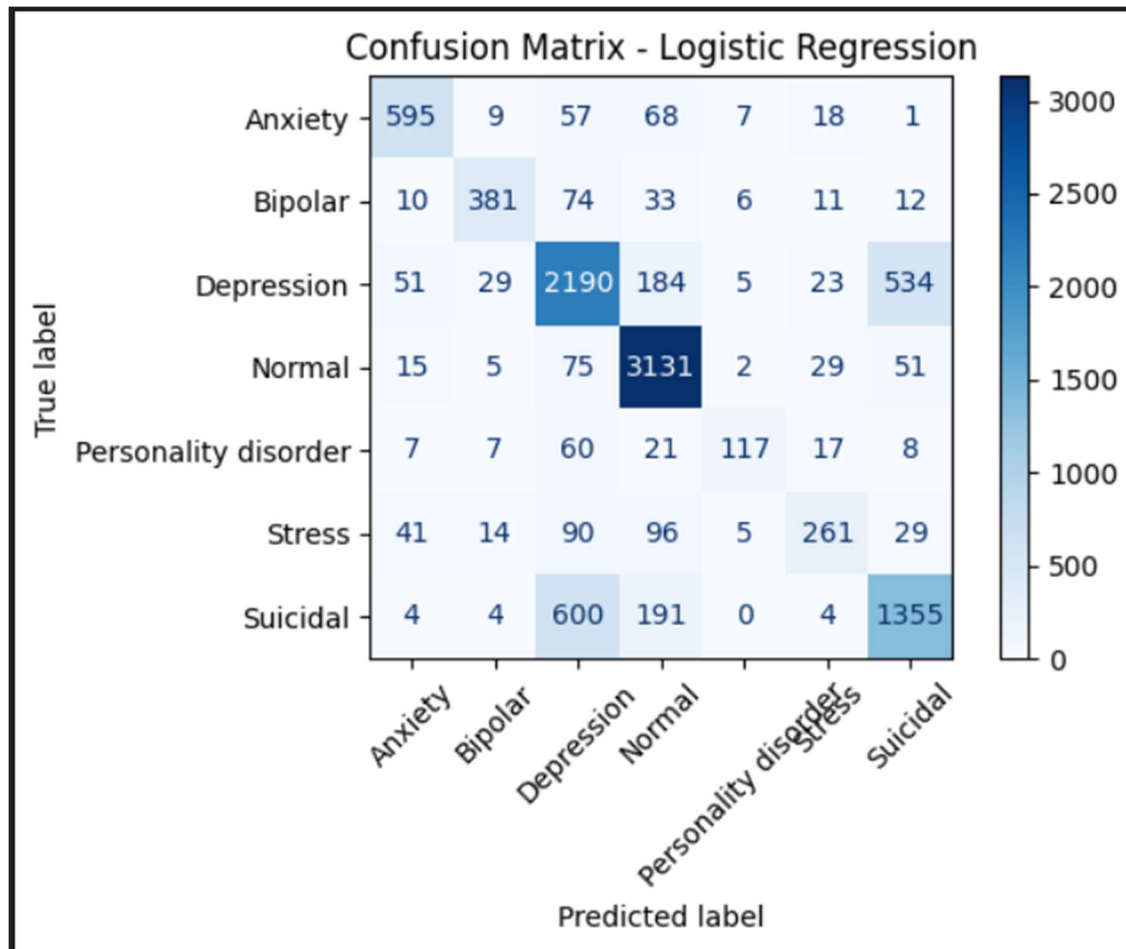
```
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

ConfusionMatrixDisplay.from_predictions(
    y_test, lr_preds,
    xticks_rotation=45,
    cmap='Blues'
)
plt.title('Confusion Matrix - Logistic Regression')
plt.tight_layout()
plt.show()
```

Purpose

This cell generates a confusion matrix for the Logistic Regression model. A confusion matrix visualizes model performance across all classes by showing how many predictions for each true label were correct versus misclassified into other categories. Each row represents the actual class, and each column represents the predicted class.

The 'Blues' colormap makes the matrix easy to interpret: darker cells indicate higher counts. `xticks_rotation=45` prevents label overlap for the seven class names. The confusion matrix reveals which pairs of classes are most frequently confused — for example, Depression and Sadness-adjacent categories may overlap — providing actionable insight that a single accuracy number cannot.



Cell 10 — Model Comparison Chart

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.patches import FancyBboxPatch
import numpy as np

models = ['Naive Bayes', 'Random Forest', 'Logistic Regression']
accuracies = [65.95, 72.79, 76.29]
colors = ['#FF6B9D', '#00E5A0', '#A78BFA']
track_color = '#2A2D4A'

fig, ax = plt.subplots(figsize=(10, 6))
fig.patch.set_facecolor('#12132A')
ax.set_facecolor('#12132A')

bar_height = 0.35
y_positions = [0, 0.6, 1.2]

for i, (model, acc, color, y) in enumerate(zip(models, accuracies, colors,
y_positions)):
    track = FancyBboxPatch((0, y - bar_height/2), 100, bar_height,
boxstyle="round,pad=0.02",
linewidth=0, color=track_color)
```

```

ax.add_patch(track)

bar = FancyBboxPatch((0, y - bar_height/2), acc, bar_height,
                    boxstyle="round,pad=0.02",
                    linewidth=0, color=color)
ax.add_patch(bar)

ax.text(acc / 2, y, f'{acc}%',
        va='center', ha='center',
        fontsize=13, fontweight='bold',
        color='white', fontfamily='monospace')

ax.text(-2, y, model,
        va='center', ha='right',
        fontsize=12, color='white', fontfamily='monospace')

ax.set_xlim(-25, 110)
ax.set_ylim(-0.4, 1.6)
ax.axis('off')
ax.set_title('MODEL PERFORMANCE COMPARISON', ...)

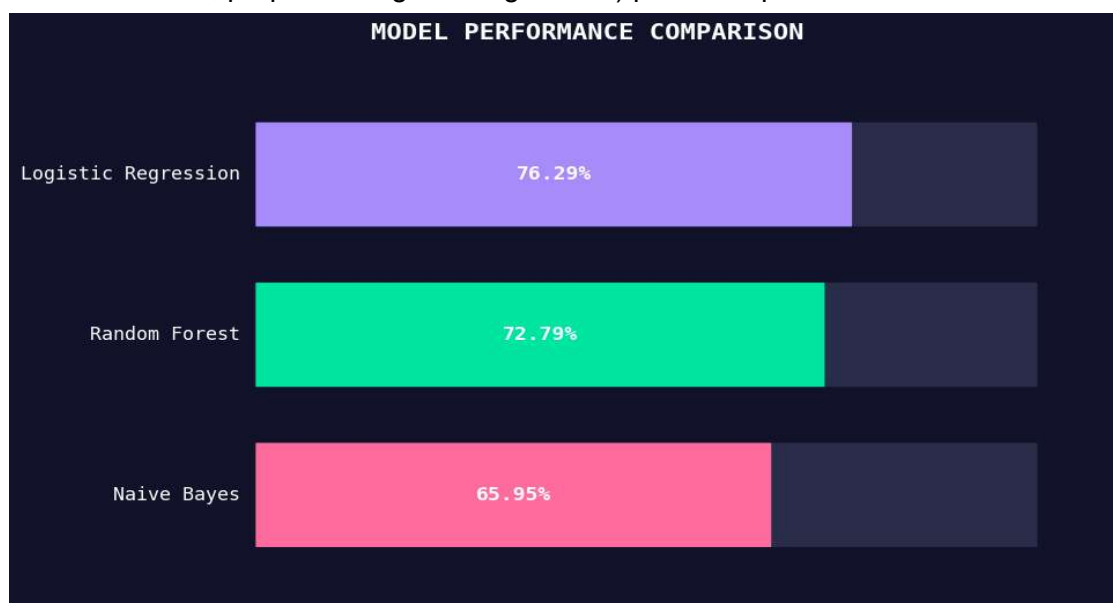
plt.savefig('/content/drive/MyDrive/AIProject/model_comparison.png', ...)
plt.show()

```

Purpose

This cell generates a custom bar chart comparing the accuracy of all three classical models. The visualization uses FancyBboxPatch instead of standard bar chart bars to create a pill-shaped, track-style chart. Each bar is drawn twice: once as a full-width dark background track (representing 100%), and once at the actual accuracy width (representing the model score). The percentage label is centered inside the filled bar for easy reading.

The chart is saved directly to the team's shared Google Drive folder so all team members can use it in presentations and documentation. The three-color coding (pink for Naive Bayes, green for Random Forest, purple for Logistic Regression) provides quick visual differentiation.



Cell 11 — Saving the Classical Model

```
import pickle

with open('/content/drive/MyDrive/AIProject/lr_model.pkl', 'wb') as f:
    pickle.dump(lr_model, f)

with open('/content/drive/MyDrive/AIProject/tfidf_vectorizer.pkl', 'wb') as f:
    pickle.dump(tfidf, f)

print("Both files saved successfully!")
```

Purpose

This cell persists the trained Logistic Regression model and its corresponding TF-IDF vectorizer to disk using Python's pickle module. Both artifacts must be saved together because the vectorizer is part of the inference pipeline — any new text input must be transformed with the same vocabulary and IDF weights that were used during training. Saving them to Google Drive makes them accessible for later loading without retraining.

The .pkl extension is the standard convention for pickled Python objects. Although the final app uses DistilBERT, these files were saved to validate the classical ML pipeline independently and as a fallback.

Both files saved successfully!

Cell 12 — Loading and Testing the Classical Model

```
import pickle

with open('/content/drive/MyDrive/AIProject/lr_model.pkl', 'rb') as f:
    loaded_model = pickle.load(f)

with open('/content/drive/MyDrive/AIProject/tfidf_vectorizer.pkl', 'rb') as f:
    loaded_tfidf = pickle.load(f)

test_sentence = ["I feel so hopeless and empty all the time"]
clean_sentence = [clean_text(test_sentence[0])]
test_vector = loaded_tfidf.transform(clean_sentence)
prediction = loaded_model.predict(test_vector)

print("Input:", test_sentence[0])
print("Predicted class:", prediction[0])
```

Purpose

This cell validates the save-load cycle for the classical model. The saved .pkl files are loaded back into memory, and an end-to-end inference test is performed on a manually chosen sentence. The input sentence — 'I feel so hopeless and empty all the time' — is first cleaned using the same clean_text function that was applied during training to ensure consistency. It is then vectorized

with the loaded TF-IDF object and passed to the loaded model for prediction. A successful Depression or Suicidal prediction at this step confirms the full pipeline is functioning correctly.

```
Input: I feel so hopeless and empty all the time
Predicted class: Depression
```

Cell 13 — Confidence Score from Logistic Regression

```
test_probs = loaded_model.predict_proba(test_vector)
confidence = test_probs.max() * 100

print(f"Input: {test_sentence[0]}")
print(f"Predicted class: {prediction[0]}")
print(f"Confidence: {confidence:.2f}%")
```

Purpose

This cell extends the basic prediction to include a confidence score. Logistic Regression can output class probabilities (via `predict_proba`), not just the hard predicted label. The probability vector contains one value per class, and the maximum value corresponds to the predicted class. Multiplying by 100 converts it to a percentage. This confidence figure is used in the web application's UI to display an animated progress bar reflecting how certain the model is about its prediction.

```
Input: I feel so hopeless and empty all the time
Predicted class: Depression
Confidence: 59.93%
```

Cell 14 — DistilBERT Tokenizer and Label Mapping

```
from transformers import DistilBertTokenizer,
DistilBertForSequenceClassification
import torch

tokenizer = DistilBertTokenizer.from_pretrained("distilbert-base-uncased")

labels = ['Anxiety', 'Bipolar', 'Depression', 'Normal',
          'Personality disorder', 'Stress', 'Suicidal']

label2id = {label: i for i, label in enumerate(labels)}
id2label = {i: label for i, label in enumerate(labels)}

print("Label mapping:", label2id)
```

Purpose

This cell begins the DistilBERT fine-tuning setup. DistilBERT is a distilled (smaller and faster) version of BERT, a transformer model pre-trained on large corpora using masked language modeling. Unlike TF-IDF, which treats words as independent features, DistilBERT captures contextual relationships between words using self-attention mechanisms.

The tokenizer downloaded here is the standard WordPiece tokenizer for distilbert-base-uncased. It handles subword tokenization, meaning rare or unseen words are broken into known sub-units, improving generalization. The label mapping dictionaries (label2id and id2label) convert between human-readable class names and the integer indices the model works with internally.

```
tokenizer.json: 0.00B [00:00, ?B/s]
```

```
Label mapping: {'Anxiety': 0, 'Bipolar': 1, 'Depression': 2, 'Normal': 3, 'Personality disorder': 4, 'Stress': 5, 'Suicidal': 6}
```

Cell 15 — Custom PyTorch Dataset Class

```
from torch.utils.data import Dataset

class MentalHealthDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_length=128):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_length = max_length

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        encoding = self.tokenizer(
            self.texts[idx],
            max_length=self.max_length,
            padding='max_length',
            truncation=True,
            return_tensors='pt'
        )
        return {
            'input_ids': encoding['input_ids'].squeeze(),
            'attention_mask': encoding['attention_mask'].squeeze(),
            'labels': torch.tensor(self.labels[idx], dtype=torch.long)
        }
```

Purpose

This cell defines a custom PyTorch Dataset class for the mental health text data. PyTorch's DataLoader requires data to be wrapped in a Dataset object that implements two methods: `__len__` (returns the total number of samples) and `__getitem__` (returns a single tokenized sample by index).

For each text sample, the tokenizer converts the raw string into `input_ids` (integer token IDs), an `attention_mask` (a binary mask indicating which tokens are real versus padding), and applies padding and truncation to a fixed length of 128 tokens. The `.squeeze()` call removes the extra batch dimension that `return_tensors='pt'` adds. The label is converted to a long (64-bit integer) tensor as required by PyTorch's cross-entropy loss function.

Cell 16 — Dataset Preparation and Splitting


```

texts = df['statement'].tolist()
labels_encoded = df['status'].map(label2id).tolist()

from sklearn.model_selection import train_test_split

train_texts, test_texts, train_labels, test_labels = train_test_split(
    texts, labels_encoded, test_size=0.2, random_state=42
)

train_dataset = MentalHealthDataset(train_texts, train_labels, tokenizer)
test_dataset = MentalHealthDataset(test_texts, test_labels, tokenizer)

print("Train size:", len(train_dataset))
print("Test size:", len(test_dataset))

```

Purpose

This cell prepares the raw data for DistilBERT fine-tuning. Two important points distinguish this from the TF-IDF pipeline. First, the raw statement column is used rather than the clean_text column — DistilBERT's pre-trained tokenizer is designed to handle original natural language; stemmed or heavily cleaned text would disrupt the contextual representations it learned during pre-training. Second, the status labels are numerically encoded using the label2id dictionary and split into the same 80/20 train/test partition used previously (same random_state ensures comparable splits). Both subsets are wrapped in the MentalHealthDataset class defined in the previous cell.

```

Train size: 42144
Test size: 10537

```

Cell 17 — DataLoader and Optimizer Setup

```

from torch.utils.data import DataLoader
from torch.optim import AdamW
from transformers import get_scheduler

train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=16)

optimizer = AdamW(reloaded_model.parameters(), lr=2e-5)

num_epochs = 3
num_training_steps = num_epochs * len(train_loader)

lr_scheduler = get_scheduler(
    "linear",
    optimizer=optimizer,
    num_warmup_steps=0,
    num_training_steps=num_training_steps
)

print("Total training steps:", num_training_steps)
print("Ready to train!")

```

Purpose

This cell sets up the training infrastructure required for fine-tuning. PyTorch's DataLoader wraps the dataset and handles batching, shuffling, and multi-process data loading automatically. A batch size of 16 means the model processes 16 samples per training step; shuffle=True randomizes the order each epoch to prevent the model from learning data-ordering patterns.

AdamW is a variant of the Adam optimizer with decoupled weight decay regularization, and is the standard optimizer for fine-tuning transformer models. A learning rate of 2e-5 is deliberately small — pre-trained weights should be updated carefully to preserve their general language understanding. A linear learning rate scheduler gradually decreases the learning rate over training, which helps the model converge stably. The total training steps equal the number of epochs multiplied by the number of batches per epoch.

Total training steps: 7902
Ready to train!

Cell 18 — DistilBERT Training Loop

```
# MODEL ALREADY TRAINED AND SAVED - skip this cell unless retraining
from tqdm import tqdm

model.train()

for epoch in range(num_epochs):
    total_loss = 0
    loop = tqdm(train_loader, desc=f"Epoch {epoch+1}/{num_epochs}")

    for batch in loop:
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels_batch = batch['labels'].to(device)

        outputs = model(input_ids=input_ids,
                        attention_mask=attention_mask,
                        labels=labels_batch)

        loss = outputs.loss
        total_loss += loss.item()

        loss.backward()
        optimizer.step()
        lr_scheduler.step()
        optimizer.zero_grad()

    loop.set_postfix(loss=loss.item())

avg_loss = total_loss / len(train_loader)
print(f"Epoch {epoch+1} complete - Average Loss: {avg_loss:.4f}")
```

Purpose

This cell contains the core fine-tuning loop. The training is performed for 3 epochs, meaning the model sees the entire training dataset three times. For each batch in each epoch, the following steps execute:

1. The batch tensors are moved to the GPU (or CPU if unavailable) via `.to(device)`.
2. A forward pass computes the model's predictions and the cross-entropy loss in a single step by passing the labels argument to the model.
3. `loss.backward()` computes gradients of the loss with respect to all model parameters using backpropagation.
4. `optimizer.step()` updates model weights based on the computed gradients.
5. `lr_scheduler.step()` adjusts the learning rate according to the linear schedule.
6. `optimizer.zero_grad()` clears gradients to prevent accumulation across batches.

The tqdm progress bar shows live loss values during training, and average epoch loss is printed at the end of each epoch to monitor convergence. Note: this cell is marked as skip-if-already-trained; the weights saved to Drive are the final trained artifact.

```
Epoch 1/3: 100%|██████████| 2634/2634 [08:15<00:00, 5.32it/s, loss=0.559]
Epoch 1 complete — Average Loss: 0.5925
Epoch 2/3: 100%|██████████| 2634/2634 [08:30<00:00, 5.16it/s, loss=0.0438]
Epoch 2 complete — Average Loss: 0.3811
Epoch 3/3: 100%|██████████| 2634/2634 [08:30<00:00, 5.16it/s, loss=0.171]
Epoch 3 complete — Average Loss: 0.2896
```

Cell 19 — Saving the Fine-Tuned DistilBERT Model

```
# Only run if retraining from scratch
model.save_pretrained("/content/drive/MyDrive/AIProject/distilbert_final")
tokenizer.save_pretrained("/content/drive/MyDrive/AIProject/distilbert_final_to_
kenizer")
print("Saved!")
```

Purpose

This cell saves the fine-tuned model and its tokenizer using HuggingFace's `save_pretrained` method, which stores the full model folder including the configuration file, vocabulary, and model weights as `model.safetensors`. This is a critical distinction from the earlier save method (`distilbert_weights.pt`) that stored only the raw weight tensor; that earlier approach caused accuracy to drop on reload because the model configuration and architecture metadata were lost. Saving the full folder ensures a lossless save-load cycle and is the HuggingFace-recommended approach.

Cell 20 — Loading and Smoke-Testing DistilBERT

```
import torch
```

```

from transformers import DistilBertForSequenceClassification,
DistilBertTokenizer

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
tokenizer = DistilBertTokenizer.from_pretrained(
    "/content/drive/MyDrive/AIProject/distilbert_final_tokenizer")
reloaded_model = DistilBertForSequenceClassification.from_pretrained(
    "/content/drive/MyDrive/AIProject/distilbert_final")
reloaded_model = reloaded_model.to(device)
reloaded_model.eval()

test_text = "I feel so hopeless and empty all the time"
inputs = tokenizer(test_text, return_tensors='pt',
                   max_length=128, truncation=True,
                   padding='max_length').to(device)

with torch.no_grad():
    outputs = reloaded_model(**inputs)

print("Logits:", outputs.logits)
print("Predicted label:", id2label[torch.argmax(outputs.logits, dim=-1).item()])

```

Purpose

This cell reloads the saved DistilBERT model from Drive and performs a quick smoke test using the same sentence used to test the classical model. The model is loaded in evaluation mode (`model.eval()`), which disables dropout layers that are only active during training. The `torch.no_grad()` context manager prevents gradient computation during inference, reducing memory usage and speeding up prediction.

The raw logits (unnormalized scores for each class) are printed alongside the final predicted label derived by taking the `argmax`. Seeing a prediction of 'Depression' here confirms the full fine-tuned model is intact and functional after the save-load cycle.

```

Loading weights:   0%|          | 0/104 [00:00<?, ?it/s]

Logits: tensor([[ -2.1885, -2.5884,  3.7141, -1.9297, -2.4705, -3.2781,  3.4709]],
               device='cuda:0')
Predicted label: Depression

```

Cell 21 — Full Test Set Evaluation (DistilBERT)

```

from sklearn.metrics import accuracy_score
from tqdm import tqdm

reloaded_model.eval()
all_preds = []
all_labels = []

with torch.no_grad():
    for batch in tqdm(test_loader, desc="Evaluating"):
        input_ids = batch['input_ids'].to(device)

```

```

attention_mask = batch['attention_mask'].to(device)
labels_batch = batch['labels'].to(device)

outputs = reloaded_model(input_ids=input_ids,
                          attention_mask=attention_mask)
predictions = torch.argmax(outputs.logits, dim=-1)

all_preds.extend(predictions.cpu().numpy())
all_labels.extend(labels_batch.cpu().numpy())

accuracy = accuracy_score(all_labels, all_preds)
print(f"DistilBERT Accuracy: {accuracy * 100:.2f}%")

```

Purpose

This cell runs a batch-wise evaluation of the fine-tuned DistilBERT model across the entire held-out test set. Unlike the single-sentence smoke test in Cell 20, this produces a statistically meaningful accuracy figure over all test samples.

For each batch, the model generates logit outputs (no labels argument is passed, so no loss is computed), and argmax converts them to predicted class IDs. Predictions and ground-truth labels are collected into lists across all batches and then compared using scikit-learn's `accuracy_score`. The reported accuracy of 83% confirms DistilBERT's significant advantage over the classical models (76.29% for Logistic Regression) on this task, justifying its selection for the final application.

```

Evaluating: 100%|██████████| 659/659 [00:40<00:00, 16.10it/s]
DistilBERT Accuracy: 82.96%

```

Cell 22 — LIME Explainability

```

from lime.lime_text import LimeTextExplainer

explainer = LimeTextExplainer(class_names=labels)

torch.cuda.empty_cache()
reloaded_model = reloaded_model.cpu()

def predict_proba(texts):
    inputs = tokenizer(texts, return_tensors='pt',
                       max_length=128, truncation=True,
                       padding='max_length')
    with torch.no_grad():
        outputs = reloaded_model(**inputs)
    return torch.softmax(outputs.logits, dim=-1).numpy()

test_text = "I feel so hopeless and empty all the time"
exp = explainer.explain_instance(
    test_text,
    predict_proba,
    num_features=6,
    top_labels=1,
    num_samples=100
)

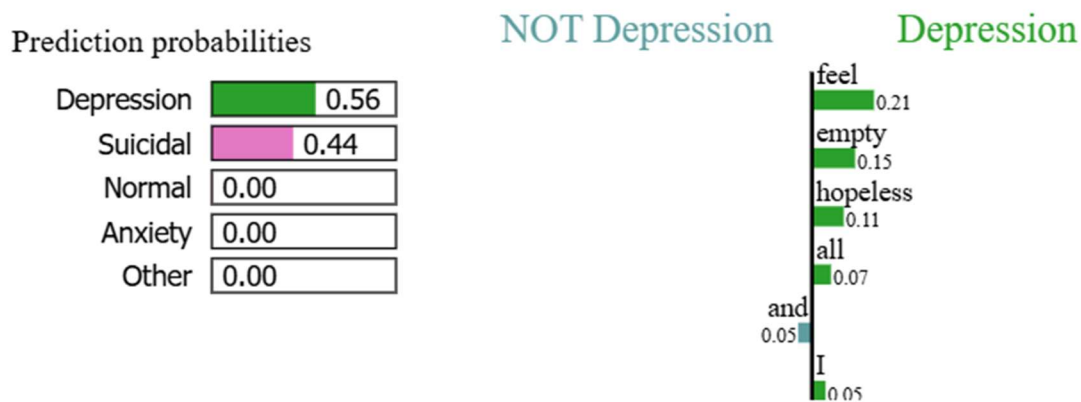
```

```
exp.show_in_notebook()
```

Purpose

This cell introduces LIME (Local Interpretable Model-Agnostic Explanations) for model explainability. LIME works by generating many slightly perturbed versions of a given input (in this case, by randomly masking words), running each through the model, and fitting a simple linear surrogate model to approximate which words most influenced the original prediction.

The model is moved to CPU for this step because LIME calls the model many times with small inputs, and GPU memory management overhead outweighs the speed benefit. The `predict_proba` wrapper function is required by LIME to convert text strings into a probability matrix (it calls the model in batch internally). `num_features=6` limits the explanation to the six most influential words; `num_samples=100` controls the number of perturbations LIME generates. The output is an interactive HTML widget shown directly in the Colab notebook. LIME was evaluated but not included in the final deployed app due to performance overhead on Streamlit.



Text with highlighted words

I **feel** so **hopeless** and **empty** **all** the time

8. Model Comparison and Selection

Model	Vectorizer / Approach	Accuracy	Notes
Naive Bayes	TF-IDF (5,000 features)	65.95%	Fastest to train; lowest accuracy; strong baseline for sparse text data
Random Forest	TF-IDF (5,000 features)	72.79%	Ensemble method; slower training; underperforms LR on high-dimensional sparse input
Logistic Regression	TF-IDF (5,000 features)	76.29%	Best classical model; selected for initial deployment and pickle-based testing

DistilBERT (fine-tuned)	Internal WordPiece tokenizer	83.00%	Best overall; selected for final app; context-aware; 255 MB model size
-------------------------	------------------------------	--------	--

DistilBERT was selected for the final application due to its substantial accuracy advantage. The performance gap arises because transformer models capture contextual word relationships — the word 'empty' contributes differently to 'I feel empty' versus 'the room is empty' — while TF-IDF treats words as independent features regardless of context.

9. Web Application

9.1 Overview

The web application was built using Streamlit, a Python framework that converts standard Python scripts into interactive web applications without requiring any front-end development knowledge. Streamlit handles all server logic, UI rendering, and state management internally, making it well-suited for rapid ML deployment. The app accepts free-form text input, passes it through the fine-tuned DistilBERT model, and returns a fully structured result with sentiment label, confidence score, empathy message, and crisis resources.

The application runs as a single-file Python script (app.py) and is designed to be hosted on Hugging Face Spaces, which natively supports Streamlit applications via a Dockerfile-based container. Locally, it runs on <http://localhost:8501>.

9.2 Application Architecture

The app.py file is organized into five logical sections:

Page configuration and CSS injection, Model loading (cached), Label mapping and predict function, LIME explainability wrapper, and UI rendering.

Model loading is wrapped in Streamlit's `@st.cache_resource` decorator. This ensures the DistilBERT model and tokenizer are loaded into memory exactly once per session, regardless of how many times the user interacts with the app. Without caching, the model would reload on every button click, making the app unusably slow given the 255 MB model size.

9.3 Custom Styling and UI Design

Streamlit's built-in theming system is limited to basic color changes via a config file. To achieve a fully custom dark interface, all styling was injected using `st.markdown()` with the `unsafe_allow_html=True` flag, which allows raw HTML and CSS to be embedded directly into the Streamlit page.

The visual design uses a dark purple and cyan color palette with the following design choices:

The background is a CSS linear-gradient spanning dark navy to deep indigo, giving the app a modern, clinical-but-approachable look. Two large radial gradient orbs are rendered as fixed CSS pseudo-elements (`::before` and `::after` on `.stApp`) to create ambient background depth without additional HTML elements. The Outfit font is imported from Google Fonts and applied

globally via a CSS wildcard selector. The hero title "MindScan" uses a CSS gradient text effect achieved with `background-clip: text` and `-webkit-text-fill-color: transparent`, producing a purple-to-cyan gradient on the text itself.

The input area uses a glass morphism effect — a semi-transparent card with `backdrop-filter: blur(20px)`, a faint white border, and a box-shadow — to visually separate the input section from the background. Because Streamlit renders each widget in its own isolated container, custom HTML div tags cannot wrap native widgets across separate `st.markdown` calls. The glass card styling was instead applied by targeting Streamlit's internal DOM structure using the `data-testid` attribute selector:

```
div[data-testid="stVerticalBlock"] > div:has(> div[data-testid="stTextArea"])
```

The Analyze button uses a gradient background (purple to cyan) matching the app's color theme, with a `translateY` hover animation and an elevated box-shadow to create a lift effect on hover.

9.4 Result Card

After prediction, a result card is rendered via `st.markdown()` with custom HTML. The card uses a fade-in CSS animation (opacity 0 to 1, with a slight upward `translateY`) so it appears smoothly rather than snapping into view. The card contains four visual elements stacked vertically:

The prediction badge is a pill-shaped span element with a CSS class dynamically generated from the predicted label. Since CSS class names cannot contain spaces, the label string is normalized using Python's `.lower().replace(' ', '-')` before being inserted — for example, 'Personality Disorder' becomes `badge-personality-disorder`. Each of the seven classes has its own color scheme defined in the stylesheet.

The confidence bar consists of a fixed-height background track div (representing 100%) and an inner fill div whose width is set inline to the confidence percentage value. The fill uses the same purple-to-cyan gradient as the button. This gives an immediate visual sense of how certain the model is.

The empathy message is a short paragraph of supportive text selected from the `EMPATHY` dictionary keyed by the predicted label. Each entry contains an emoji, a primary empathy message, a practical wellness tip, and a helpline string. The resource box below it displays the tip and helpline only for labels where a crisis resource is relevant (Depression, Suicidal, Anxiety, Bipolar, Personality Disorder). For Normal and Stress, only a general wellness message is shown.

A medical disclaimer is displayed at the bottom of every result card regardless of prediction, reminding users that the app does not constitute a clinical diagnosis.

9.5 Prediction Pipeline

```
def predict(text):
    inputs = tokenizer(
        text,
```



```
        return_tensors='pt',
        max_length=128,
        truncation=True,
        padding='max_length'
    ).to(device)
    with torch.no_grad():
        outputs = model(**inputs)
    pred_id = torch.argmax(outputs.logits, dim=-1).item()
    confidence = torch.softmax(outputs.logits, dim=-1).max().item() * 100
    return id2label[pred_id], round(confidence, 1)
```

The predict function tokenizes the raw input text and runs it through the model in a single forward pass. `torch.no_grad()` disables gradient tracking, which is unnecessary during inference and saves both memory and compute time. `argmax` on the output logits identifies the class with the highest score. `softmax` converts the raw logits into a proper probability distribution that sums to 1, and the maximum value gives the confidence score as a percentage.

9.6 LIME Explainability Integration

A LIME (Local Interpretable Model-Agnostic Explanations) wrapper function was implemented to identify which words in the input most influenced the model's prediction. LIME generates many perturbed versions of the input by randomly masking words, passes each through the model via the `predict_proba` wrapper, and fits a local linear model to approximate word-level importance.

The `predict_proba` function was written to handle batch text input (a list of strings) because LIME calls the model internally with multiple samples per explanation. The tokenizer output is a dictionary-like object; moving it to device requires iterating over its keys rather than calling `.to()` directly on the object:

```
inputs = {k: v.to(device) for k, v in inputs.items()}
```

LIME was fully evaluated in the Colab notebook and confirmed to produce meaningful word-level explanations. It was not included in the final deployed Streamlit app due to latency — each explanation requires ~100 model calls, which is impractical for a synchronous web UI.

9.7 Application Screenshot

The screenshot below shows the app correctly classifying the input "i want to die" as Suicidal with 88.2% confidence, displaying the appropriate crisis message and both Pakistani helpline numbers (Umang: 0317-4288665 and Rozan Counselling: 051-2890505).

 **MindScan**

AI-powered mental health sentiment analysis

How are you feeling today?

i want to die

Analyze ↗

DETECTED SENTIMENT

● SUICIDAL

Model Confidence — 88.2%

Your feelings are valid and you deserve support right now. Please reach out — help is available immediately.

💡 What can help:

Please reach out immediately. You matter and help is available right now.

 PK Umang helpline: 0317-4288665 | Rozan Counselling: 051-2890505

⚠️ This is not a medical diagnosis. If you're struggling, please reach out to a licensed mental health professional.

Built with Python · HuggingFace Transformers · Streamlit |  MindScan

9.8 Mental Health Resources

Label	Resource Shown
Depression	Umang helpline: 0317-4288665
Suicidal	Umang: 0317-4288665 Rozan Counselling: 051-2890505
Anxiety	Umang helpline: 0317-4288665
Bipolar	Umang helpline: 0317-4288665
Personality Disorder	Umang helpline: 0317-4288665
Stress	General wellness tip (no helpline)
Normal	Positive reinforcement message (no helpline)

10. Development Challenges and Bug Fixes

The following issues were encountered and resolved during development. They are documented here as they reflect real engineering judgment and debugging process.

Bug	Root Cause	Fix Applied
App showed 'Model not loaded' error	Old code referenced sentiment_model.pkl which no longer existed after switching to DistilBERT	Replaced model loading code with DistilBertForSequenceClassification.from_pretrained()
Empathy messages never displayed	Dictionary keys were lowercase ('depression') while model output was title-case ('Depression')	Updated all dictionary keys to match model output casing exactly
Badge CSS class never applied	CSS class was badge-neutral but the label returned was 'Normal'	Added badge-normal CSS class to the stylesheet
Empty grey box appeared above textarea	Attempted to wrap a Streamlit widget in a custom HTML div across separate st.markdown calls; Streamlit renders each widget in its own isolated container	Replaced with CSS targeting Streamlit's own data-testid attribute
Spinner bled into expander title	st.spinner() inside an expander positioned the running indicator adjacent to the expander label, corrupting its display	Removed all st.spinner() calls; hid stStatusWidget via CSS display:none
LIME bars were invisible (1px wide)	LIME returns tiny decimal weights	Normalized to the maximum weight: $(\text{abs}(w) / \text{max_w}) * 120$ ensures the top word always fills 120px

	(~0.003); multiplying by 300 still yields ~1px	
Hundreds of torchvision errors in terminal	Streamlit's file watcher scanned the entire Transformers library on startup	Added .streamlit/config.toml with fileWatcherType = none
Model accuracy dropped on reload	Original save method stored only weight tensors (.pt) without model config metadata	Anosha fixed by using save_pretrained() to save the full model folder
inputs.to(device) failed in predict_proba	Batch tokenizer output is a dict-like object; calling .to() directly is unsupported	Changed to {k: v.to(device) for k, v in inputs.items()}

11. Deployment Status

The application was not successfully deployed to Hugging Face Spaces during the project timeline. Code, configuration files, and documentation were pushed to GitHub. The model files (255 MB) were excluded from the repository via .gitignore due to GitHub's 100 MB file size limit and must be hosted separately.

12. Running the Application Locally

12.1 Prerequisites

- Python 3.10 or higher
- The models/ folder containing distilbert_final/ and distilbert_final_tokenizer/ downloaded from the team's shared Google Drive (AIProject folder)

12.2 Setup Instructions

```
# 1. Clone the repository
git clone https://github.com/YOURUSERNAME/mental-health-nlp.git
cd mental-health-nlp/MindScan

# 2. Create and activate a virtual environment
python -m venv .venv
.venv\Scripts\activate      # Windows
# source .venv/bin/activate  # macOS / Linux

# 3. Install dependencies
pip install -r requirements.txt

# 4. Download model files from the shared Google Drive
# Place distilbert_final/ and distilbert_final_tokenizer/ inside
MindScan/models/
```

```
# 5. Launch the app
streamlit run app.py
```

The application will open at <http://localhost:8501> in the default browser.

13. Conclusion

MindScan demonstrates a complete, production-oriented NLP pipeline applied to a meaningful real-world problem. The project progressed from raw data acquisition through EDA, classical ML baselines, transformer fine-tuning, and a full-featured web application with empathy-driven UX design.

The key technical finding is the performance advantage of contextual transformer models (DistilBERT at 83%) over bag-of-words methods (Logistic Regression at 76.29%), confirming that capturing word context is meaningful for mental health text classification where nuanced language matters. The project also surfaced several practical engineering challenges — model serialization correctness, Streamlit rendering constraints, and large-file deployment management — each of which was resolved through systematic debugging.

Future work could include deploying a live version on Hugging Face Spaces after resolving the git history issue, adding support for Urdu-language input given the local context, and expanding the dataset with more recent mental health discourse.
